

DEVELOPER CASE STUDY

End-to-End Claims Processing Solution

Java and AWS architecture, integration and sample implementation

Prepared by

Sethu Budaza

Technology stack

Java 21, Spring Boot 3.x, Gradle, PostgreSQL and AWS

Reliable

Durable intake, safe retries and payment reconciliation

Clear ownership

Each system keeps control of its own business data

SLA-aware

Urgent claims receive reserved processing capacity

Document overview

Document	End-to-End Claims Processing Solution
Author	Sethu Budaza
Version	1.0
Status	Case study response
Selected stack	Java 21, Spring Boot 3.x, Gradle, PostgreSQL and AWS
Scope	Architecture, workflow, system boundaries, APIs, reliability, project structure, sample code and dummy endpoints

Purpose

This document explains how I would connect the Channel System, Claims UI, Client Registry, Policy Manager and Payment System. The aim is to process claims safely from submission to payment, while keeping the design practical enough to build and demonstrate.

My main design choices

- Accept the claim quickly, then continue the work in the background.
- Make the Claims Platform the only writer of claim workflow status.
- Use a durable workflow for retries, waits, timeouts and analyst review.
- Prevent duplicate claims and duplicate payments with idempotency.
- Keep a full audit history and make SLA risk visible to operations.

What this design avoids

- A long synchronous request across several systems.
- Direct database access between systems.
- A distributed transaction across Claims, Client, Policy and Payment.
- Blind retries after an uncertain payment result.
- Business rules copied into every channel or integration adapter.

Contents

Executive summary	4
1. Problem understanding and goals	5
2. Proposed solution	6
3. System context and boundaries	7
4. AWS logical architecture	8
5. End-to-end system interaction	9
6. Claim lifecycle and business rules	10
7. API and event contracts	11
7. API and event contracts - continued	12
8. Data, consistency and concurrency	13
9. Reliability and failure handling	14
10. SLA management and urgent claims	15
11. Security, privacy and audit	16
12. Observability and operations	17
13. Integration changes and migration	18
14. Project structure and engineering standards	19
15. Java implementation - API layer	20
15. Java implementation - transaction and outbox	21
15. Java implementation - domain and integrations	22
16. Dummy endpoints and local demonstration	23
17. Testing strategy	24
18. CI/CD and deployment	25
19. Assumptions, questions and trade-offs	26
20. Delivery plan, AI use and conclusion	27

Executive summary

I propose a Claims Processing Platform that owns the claim workflow from the moment a claim is accepted until payment is confirmed.

The web channel sends claim JSON to a versioned API. The API validates the request, writes the claim and an outbox event in one database transaction, and returns `202 Accepted`. The caller does not wait for the Client Registry, Policy Manager or Payment System.

A durable workflow then validates the client, checks policy and benefit eligibility, pauses for analyst review when required, and creates one idempotent payment request after approval. The Payment System confirms the result through a signed callback. A reconciler checks payments that remain pending or have an uncertain network result.

The production design uses API Gateway, ECS Fargate, Step Functions, EventBridge, SQS and Aurora PostgreSQL. The case study code stays smaller: a modular Spring Boot application, three external-system stubs, Docker Compose, and ports that can later be connected to AWS services.

The outcome I care about

Once the intake API has accepted a claim, the claim must not disappear. A temporary failure may slow the process down, but it must not create a duplicate claim, a duplicate payment or an unclear status.

How the design shows senior engineering judgement

Area	Decision	Why it matters
Ownership	One writer for claim workflow state	Prevents conflicting status updates across systems.
Reliability	Outbox, idempotency and reconciliation	Handles failures at the points where real systems usually break.
Operations	SLA timers, alerts, DLQs and runbooks	Makes the solution supportable after deployment.
Delivery	Modular first, extract services only when needed	Keeps the case study buildable without hiding production concerns.
Change	Phased migration with shadow traffic	Reduces risk while existing claims continue to be processed.

1. Problem understanding and goals

1.1 What the current picture tells me

The Claims UI and API currently sit in the middle of the process, but the surrounding systems are not connected. The Channel System can produce JSON, the Client Registry owns client information, the Policy Manager owns policy and benefit rules, and the third-party Payment System executes payments.

The new solution therefore needs to connect the process without taking ownership away from those systems. The Claims Platform may store the evidence returned by an external system, but it should not become the master source for client, policy or settlement data.

1.2 Business goals

- Accept claims reliably from the web channel.
- Validate the client through the Client Registry.
- Check policy, plan and benefits through Policy Manager.
- Support straight-through and analyst-assisted decisions.
- Start payment only after approval.
- Expose one clear claim status.
- Prioritise urgent claims such as death claims.
- Keep a complete audit history.

1.3 Non-functional goals

Quality	Design goal
Durability	No claim is lost after the intake transaction commits.
Availability	Claim intake stays available during a temporary downstream outage.
Consistency	Valid state transitions, optimistic locking and idempotent commands protect business outcomes.
Security	Least privilege, encryption and minimum sensitive-data storage.
Operability	Metrics, tracing, alerts, queues, reconciliation and clear recovery steps.
Maintainability	Clear modules, versioned contracts and automated tests.

Open business point

The case study says the service must meet defined SLAs, but it does not give the durations. I would store SLA rules as configuration and confirm the actual values with the claims business owner before production.

2. Proposed solution

2.1 Processing model

- 1 Submit.** The Channel System sends claim JSON, an external reference and an idempotency key.
- 2 Accept.** The Claims API validates the request, stores the claim and outbox event, and returns `202 Accepted`.
- 3 Validate.** The workflow checks the client and policy through dedicated adapters.
- 4 Decide.** The workflow rejects the claim, routes it to an analyst, or approves it.
- 5 Pay.** The platform creates one idempotent payment request and stores the payment reference.
- 6 Confirm.** A signed callback or reconciliation query records the final payment result.

2.2 Why the intake API is asynchronous

I would not make the web form wait while several downstream systems respond. A client or policy system may be slow or temporarily unavailable. That should delay the workflow, not cause the original submission to be lost or retried in an unsafe way.

2.3 Why I use orchestration

The claim has an ordered, long-running lifecycle. It may wait for an analyst, retry a temporary failure, or pause while payment is pending. Step Functions makes that state visible and gives the team one place to define timeouts and retry rules. Events are still useful for notifications, reporting and other side effects, but they do not replace the main claim workflow.

Principle	How it is applied
Single owner	The Claims Platform is the only writer of claim workflow status.
Clear boundaries	Client, policy and payment systems remain authoritative.
Safe delivery	Commands and events can be handled more than once without changing the final result.
Explicit workflow	Business states and technical progress are kept separate.
Build in layers	The case study starts as a modular application and preserves extraction points.

3. System context and boundaries

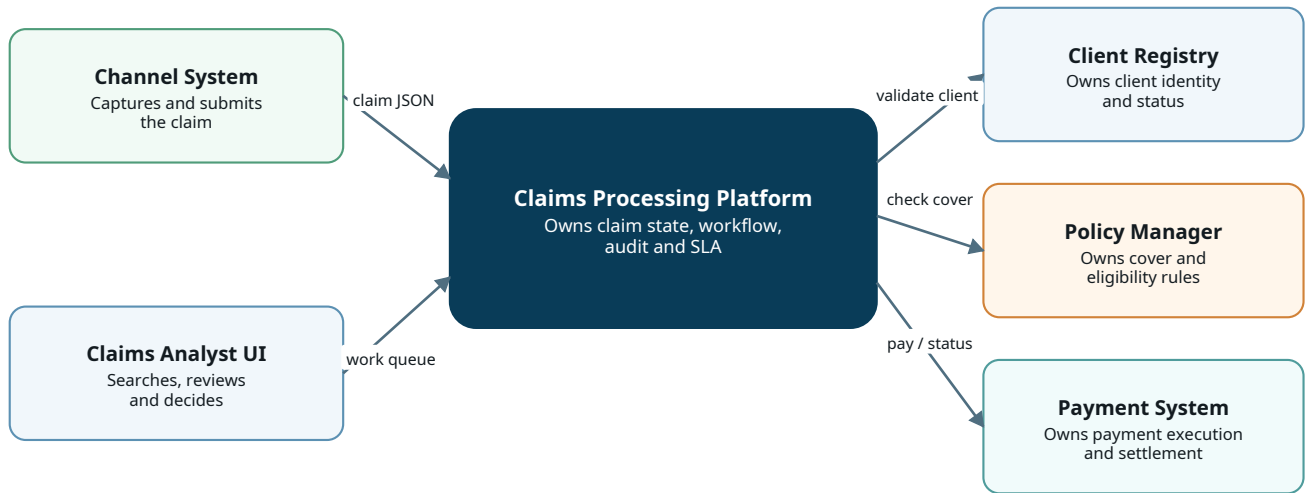


Figure 1. Proposed system context and ownership boundaries.

3.1 Boundary responsibilities

System	Owns	Must not own
Channel System	Claim capture and channel submission reference	Claim decision, workflow rules or payment execution
Claims Platform	Claim lifecycle, workflow, SLA, audit, analyst decision and integration coordination	Master client, policy or settlement data
Claims Analyst UI	Analyst interaction and work views	Direct integration with Client Registry, Policy Manager or Payment System
Client Registry	Client identity, status and client validation decision	Claim eligibility or payment state
Policy Manager	Policy, benefit, cover and eligibility decision	Claim workflow or payment settlement
Payment System	Payment execution, settlement and final payment status	Claim approval or policy eligibility

One writer for claim status

Other systems return decisions and references. They do not update claim workflow status directly. This removes a common source of race conditions and unclear ownership.

4. AWS logical architecture

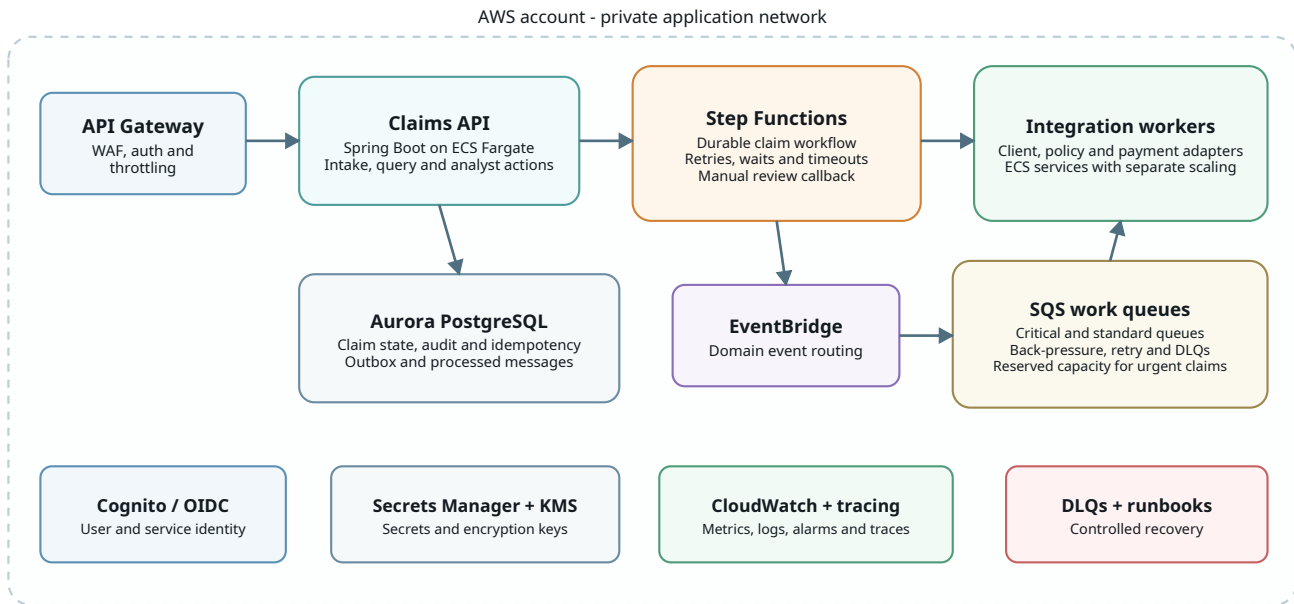


Figure 2. Production target on AWS.

4.1 Service mapping

Concern	AWS service	Reason
Public API	API Gateway + WAF	Authentication, throttling and controlled exposure.
Application runtime	ECS Fargate	Runs Spring Boot services without managing servers.
Workflow	Step Functions Standard	Long-running state, waits, retries and manual review.
Database	Aurora PostgreSQL	Transactions, constraints, audit queries and outbox support.
Work buffering	SQS	Back-pressure, retry, dead-letter handling and separate priority lanes.
Event routing	EventBridge	Publishes domain events without coupling consumers.
Operations	CloudWatch + OpenTelemetry	Logs, metrics, alarms and end-to-end traces.

Availability: The production setup uses multiple Availability Zones for ECS, Aurora and queues. The case study can run locally with Docker Compose and mock services.

5. End-to-end system interaction

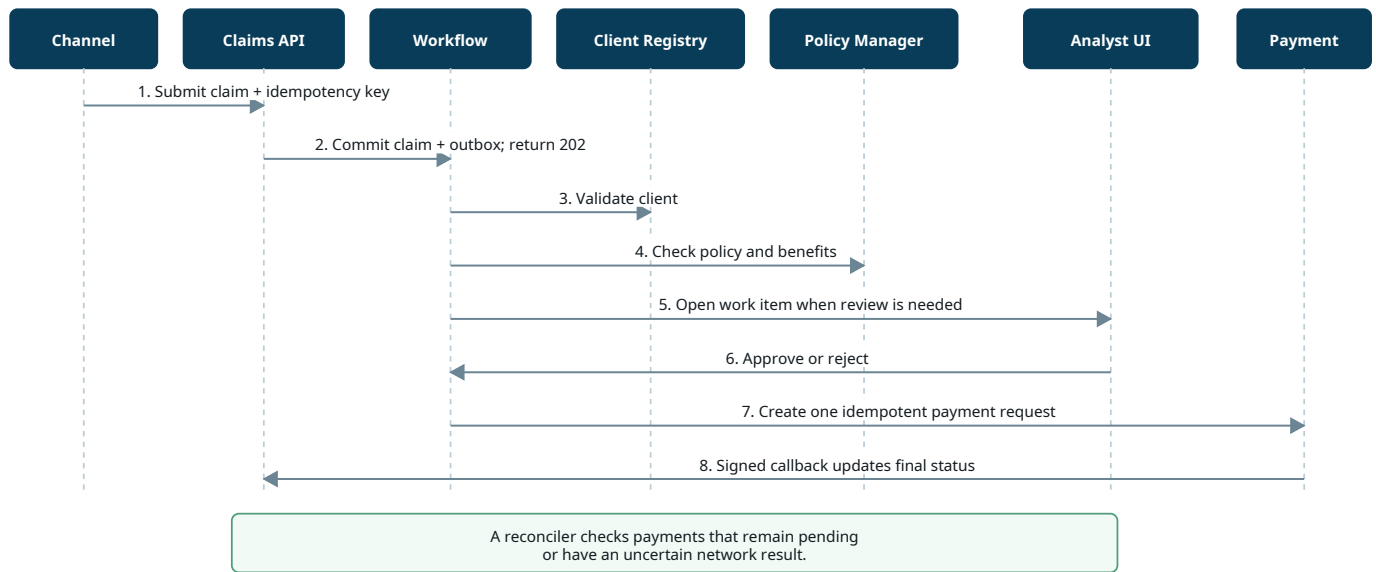


Figure 3. Main processing sequence.

5.1 Manual review

When a claim needs review, the workflow creates a work item and waits for a callback token. The token is mapped to the claim, not shown in the browser, and completed only through an authorised API. The analyst sees the client and policy results, records a reasoned decision, and the workflow continues from the same point.

5.2 Failure path

A temporary technical failure is not a business rejection. The workflow records the failed step, retries only when it is safe, and raises an operational alert when the retry limit is reached. The claim stays in a clear state such as **VALIDATING** with a technical sub-status, rather than being rejected by mistake.

Payment rule

After a payment request may have reached the third party, I would query by idempotency key before retrying. A network timeout is not proof that payment failed.

6. Claim lifecycle and business rules

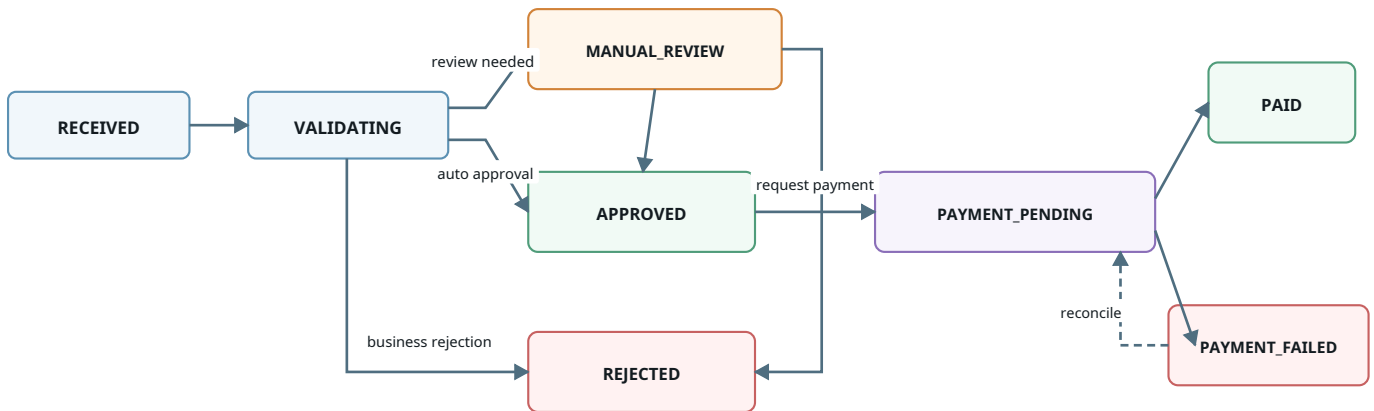


Figure 4. Proposed business state model.

6.1 State meanings

State	Meaning	Typical next states
RECEIVED	The claim is stored and acknowledged.	VALIDATING
VALIDATING	Client and policy checks are running or waiting for a safe retry.	MANUAL_REVIEW, APPROVED, REJECTED
MANUAL_REVIEW	An authorised analyst decision is required.	APPROVED, REJECTED
APPROVED	All required decisions have passed.	PAYMENT_PENDING
PAYMENT_PENDING	The payment request is accepted and waiting for a terminal result.	PAID, PAYMENT_FAILED
PAID	Payment completion is confirmed.	Terminal
REJECTED	A business rule or analyst decision rejected the claim.	Terminal

6.2 Important invariants

- A claim cannot enter `PAYMENT_PENDING` unless it is approved.
- A payment request uses a stable key such as `claim:{claimId}:payment:v1`.
- The same terminal callback is an idempotent no-op.
- A transition records the actor, reason, old state, new state and correlation identifiers.
- Technical step progress is stored separately from the business state.

7. API and event contracts

7.1 Contract rules

- Use versioned paths such as `/api/v1`.
- Require an idempotency key for commands that may create a financial or workflow effect.
- Use `application/problem+json` for errors.
- Use stable reason codes, not only free text.
- Publish OpenAPI definitions and validate requests at the boundary.
- Propagate correlation and causation identifiers.

7.2 Claims API

Method and path	Purpose	Response
POST <code>/api/v1/claims</code>	Submit a new claim	202 Accepted
GET <code>/api/v1/claims/{claimId}</code>	Return current status, priority and SLA	200 OK
GET <code>/api/v1/claims/{claimId}/timeline</code>	Return the authorised audit timeline	200 OK
GET <code>/api/v1/claims</code>	Search analyst work with pagination	200 OK
POST <code>/api/v1/claims/{claimId}/decisions</code>	Record an analyst decision	200 OK
POST <code>/internal/v1/payment-status-events</code>	Receive signed payment status	202 Accepted

Example: submit a claim

TEXT

```
POST /api/v1/claims
Idempotency-Key: 71825ac3-2654-4734-bd43-5f9969b89323
X-Correlation-Id: 6b8b65d4-9795-4dbc-9998-ef5d48dd2467
Content-Type: application/json

{
  "externalReference": "WEB-000123",
  "claimType": "DEATH",
  "clientId": "CLIENT-10542",
  "policyNumber": "POL-847563",
  "incidentDate": "<YYYY-MM-DD>",
  "submittedAt": "<ISO-8601 timestamp>",
  "amount": {
    "currency": "ZAR",
    "value": 250000.00
  },
  "channel": "WEB"
}
```

Example: accepted response

TEXT

```
HTTP/1.1 202 Accepted
Location: /api/v1/claims/4d5b34b1-4d7e-47f4-8175-bcd7658b2755

{
  "claimId": "4d5b34b1-4d7e-47f4-8175-bcd7658b2755",
  "externalReference": "WEB-000123",
  "status": "RECEIVED",
  "priority": "CRITICAL",
  "statusUrl": "/api/v1/claims/4d5b34b1-4d7e-47f4-8175-bcd7658b2755"
}
```

7. API and event contracts - continued

7.3 Idempotency behaviour

Situation	Required behaviour
New key and valid request	Create the claim, store the request hash and return the accepted result.
Existing key with the same request	Return the original result without starting another workflow.
Existing key with a different request	Return <code>409 Conflict</code> .
Concurrent requests with the same external reference	A database unique constraint chooses one winner; the other request returns the existing claim or a conflict.

application/problem+json

JSON

```
{
  "type": "about:blank",
  "title": "Idempotency key conflict",
  "status": 409,
  "code": "CLAIM_IDEMPOTENCY_CONFLICT",
  "detail": "The key was already used with a different request.",
  "correlationId": "6b8b65d4-9795-4dbc-9998-ef5d48dd2467"
}
```

7.4 Event envelope

Every event has a unique ID, version, aggregate ID, correlation ID and causation ID. Consumers store processed event IDs so that delivery can safely happen more than once.

Domain event envelope

JSON

```
{
  "eventId": "1c885c71-ce4a-43ee-91a4-366e1ff42d9f",
  "eventType": "claim.payment.completed",
  "eventVersion": 1,
  "aggregateId": "4d5b34b1-4d7e-47f4-8175-bcd7658b2755",
  "correlationId": "6b8b65d4-9795-4dbc-9998-ef5d48dd2467",
  "causationId": "PAY-852197",
  "occurredAt": "<ISO-8601 timestamp>",
  "data": {
    "paymentReference": "PAY-852197",
    "status": "COMPLETED"
  }
}
```

8. Data, consistency and concurrency

8.1 Core records

Record	Purpose	Important controls
<code>claim</code>	Current claim business state	Unique external reference, version column, SLA due time
<code>claim_event</code>	Append-only business and audit timeline	Previous state, new state, actor, reason and correlation IDs
<code>idempotency_record</code>	Returns the original result for a retried command	Client/channel + key uniqueness, request hash
<code>integration_attempt</code>	Tracks calls to external systems	Attempt number, result, error class and duration
<code>analyst_decision</code>	Stores authorised manual decisions	Analyst ID, reason code, comment and timestamp
<code>payment_attempt</code>	Tracks the payment command and reference	Stable idempotency key and payment reference
<code>outbox_event</code>	Publishes events after the database commit	Published flag and attempt count
<code>processed_message</code>	Deduplicates inbound events	Consumer + message ID unique key

8.2 Why there is no distributed transaction

A single transaction cannot safely span Aurora, Client Registry, Policy Manager and a third-party payment provider. I use local transactions, explicit state, idempotent commands and reconciliation instead.

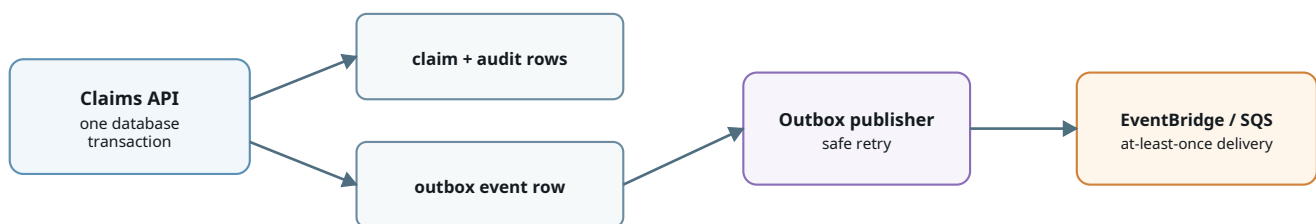


Figure 5. The claim and outbox record commit together; publication can be retried later.

8.3 Concurrency

An optimistic lock version protects the claim row. If two updates race, one update receives a conflict, reloads the claim and re-checks whether the requested transition is still valid. Database unique constraints remain the final protection for duplicate submissions and processed events.

9. Reliability and failure handling

9.1 Failure decisions

Failure	Design response
Channel retries claim submission	Return the same claim result through the idempotency record.
Database commits but event publication fails	The outbox publisher retries until the event is published.
Client Registry times out	Retry transient errors with backoff and jitter; keep the claim pending.
Policy Manager returns a business rejection	Do not retry. Record the stable reason code and rule version.
Payment request result is uncertain	Query by idempotency key before another payment attempt.
Payment callback is duplicated	Deduplicate by event ID and apply an idempotent state transition.
Events arrive out of order	Check event sequence/version and quarantine an unsafe event.
Message repeatedly fails	Move it to a DLQ, alert operations and follow a tested runbook.
Workflow is stuck	A watchdog finds stale steps and raises an alert before the SLA is missed.

9.2 Retry policy

Category	Automatic retry?	Examples
Transient technical	Yes, with a limit	Timeout, connection reset, HTTP 429, 502, 503, 504
Permanent technical	No	Invalid contract, unsupported response, authentication failure
Business result	No	Client inactive, policy not active, benefit excluded
Uncertain payment result	Reconcile first	Connection lost after the payment request was sent

Practical recovery rule

A DLQ message should never be replayed blindly. Operations must know whether the work is safe to repeat, whether a compensating action is needed, and how the claim will be audited.

10. SLA management and urgent claims

10.1 Configurable SLA model

At intake, the platform calculates priority, target completion time and escalation checkpoints from versioned SLA rules. The selected rule and calculated due time are stored on the claim so that later rule changes do not change an existing commitment.

Claim category	Priority	Processing treatment
Death and other defined urgent claims	CRITICAL	Dedicated queue, reserved workers, earlier warnings and tighter escalation.
High-value or vulnerable-customer claims	HIGH	Higher scheduling weight and earlier review alert.
Standard claims	NORMAL	Normal queue and normal worker pool.

10.2 Preventing priority starvation

Critical claims need real capacity, not only a database flag. I would reserve a small worker pool for urgent claims and let remaining workers serve both queues. This protects urgent work without permanently blocking standard work.

10.3 Operational measures

Measure	What it tells us
Intake acknowledgement latency	Whether the channel receives a quick and reliable response.
Oldest claim by priority	Whether critical or standard work is waiting too long.
Time spent in each workflow step	Where the process is slowing down.
Claims close to SLA breach	What operations must act on now.
Payment pending age	Whether reconciliation or third-party support is needed.

Senior design point

The system should alert before an SLA is missed. Reporting only the final breach is too late for operations to help the client.

11. Security, privacy and audit

11.1 Security controls

Area	Control
Channel authentication	OAuth 2.0 / OpenID Connect, scoped access tokens, WAF and throttling.
Analyst authorisation	Role and claim-scope checks such as analyst, supervisor and operations roles.
Service identity	IAM roles and short-lived credentials instead of static cloud keys.
Data in transit	TLS for every connection; private networking where possible.
Data at rest	KMS encryption for Aurora, queues, logs, object storage and backups.
Secrets	AWS Secrets Manager with rotation and restricted access.
Payment callback	Signed payload, timestamp or nonce checks, and replay protection.
Abuse protection	Input limits, schema validation, rate limits and safe error responses.

11.2 Privacy by design

- Store references and decision evidence instead of copying full client and policy records.
- Mask identity and bank details in logs and normal support screens.
- Use separate permissions for claim data, integration payloads, documents and backups.
- Apply retention rules to raw payloads and audit records.
- Record read access to sensitive claim information.
- Apply the organisation's privacy and regulatory policy, including POPIA where relevant.

11.3 Audit record

The audit timeline is append-only at application level. It records who changed the claim, the old and new state, the reason, the rule or policy version, and the related correlation IDs. A corrected decision is a new event; history is not overwritten.

12. Observability and operations

12.1 Correlation and tracing

The same correlation ID follows the request from the Channel System through the API, workflow, queues and adapters. Each event also records causation and event IDs. This makes a full claim path visible even when work is asynchronous.

12.2 Dashboards

Dashboard	Representative measures
Business flow	Claims received, approved, rejected, paid and waiting by claim type and priority.
SLA	Oldest claim by queue, percentage within SLA, warning count and breach count.
Integrations	Success rate, latency, retry rate, circuit-breaker state and contract errors.
Payment	Requests sent, pending age, failures, uncertain outcomes and reconciliation backlog.
Platform	API latency, error rate, ECS health, Aurora capacity, queue depth and DLQ messages.

12.3 Alerts and runbooks

- A critical claim reaches a warning threshold before its SLA.
- A queue's oldest message exceeds its expected age.
- A payment remains pending beyond the reconciliation threshold.
- A downstream system's error rate or latency rises sharply.
- An outbox event is not published within the expected time.
- A DLQ receives a message.

Operational ownership

Every alert must point to a runbook that says how to investigate, when retry is safe, when to reconcile, and who owns the next action. Alerts without actions create noise, not reliability.

13. Integration changes and migration

13.1 Recommended changes

System	Recommended change
Channel System	Send an external reference, idempotency key and correlation ID; handle <code>202 Accepted</code> and use the status endpoint.
Existing Claims UI	Use the Claims Platform for searches and decisions; remove direct calls to client, policy or payment systems.
Existing Claims API	Either evolve into the Claims Platform or place a façade in front during migration. Ensure only one component writes claim workflow state.
Client Registry	Expose a versioned validation API with a validation reference, valid flag, client status and stable reason codes.
Policy Manager	Expose an eligibility API with policy status, benefit result, review flag, available amount, exclusions, waiting-period result and rule version.
Payment System	Support idempotent payment creation, status lookup by idempotency key, stable references and signed sequenced callbacks.
All systems	Publish OpenAPI contracts, propagate correlation IDs, define timeouts, expose health endpoints and support contract testing.

13.2 Safe migration

- 1 Agree ownership, API contracts, state names and reason codes.
- 2 Build the intake API, persistence, idempotency and outbox first.
- 3 Connect external-system stubs, then real adapters one at a time.
- 4 Run the new workflow in shadow mode for non-financial decisions.
- 5 Move analyst work queues and status reads to the new platform.
- 6 Enable payment for a controlled claim group with reconciliation and support cover.
- 7 Retire duplicate orchestration after data and operational checks pass.

Why shadow mode helps

It lets the team compare the new validation and routing decisions with the existing process before the new platform is allowed to create a payment.

14. Project structure and engineering standards

Repository layout

TEXT

```
claims-processing-platform/
├── applications/
│   ├── claims-service/
│   ├── mock-client-registry/
│   ├── mock-policy-manager/
│   └── mock-payment-system/
├── modules/
│   ├── claim-domain/
│   ├── claim-application/
│   ├── claim-persistence/
│   ├── claim-web/
│   └── claim-integrations/
├── infrastructure/
│   ├── cdk/
│   ├── step-functions/
│   ├── docker/
│   └── monitoring/
├── docs/
│   ├── architecture.md
│   ├── assumptions.md
│   ├── api/
│   └── decisions/
├── examples/
├── build.gradle
├── settings.gradle
└── docker-compose.yml
```

14.1 Package shape

```
com.company.claims
  claim/domain
  claim/application
  claim/port/in
  claim/port/out
  claim/adapters/in/web
  claim/adapters/out/persistence
  integration/clientregistry
  integration/policymanager
  integration/payment
  security observability shared
```

14.2 Engineering standards

- Small domain-focused methods.
- Immutable request and response records.
- Constructor injection.
- No controller-to-repository shortcut.
- Ports around external systems.
- Structured errors and reason codes.
- Clock and ID generators injected for tests.
- Architecture rules enforced with ArchUnit.
- OpenAPI and event schemas versioned.
- README covers local run and failure demos.

The first implementation is a modular application because that is easier to build, test and operate for this case study. The module boundaries are deliberate, so a worker or integration can be extracted later without rewriting the domain.

15. Java implementation - API layer

The examples use normal IntelliJ-style Java formatting: four-space indentation, braces on the same line, short methods and line breaks where arguments would become hard to read.

SubmitClaimRequest.java

JAVA

```
package com.company.claims.claim.api;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.PastOrPresent;

import java.math.BigDecimal;
import java.time.Instant;
import java.time.LocalDate;

public record SubmitClaimRequest(
    @NotBlank String externalReference,
    @NotNull ClaimType claimType,
    @NotBlank String clientId,
    @NotBlank String policyNumber,
    @NotNull @PastOrPresent LocalDate incidentDate,
    @NotNull Instant submittedAt,
    @Valid @NotNull MoneyRequest amount,
    @NotNull ClaimChannel channel
) {
    public record MoneyRequest(
        @NotBlank String currency,
        @NotNull BigDecimal value
    ) {
    }
}
```

ClaimIntakeController.java

JAVA

```
@RestController
@RequestMapping("/api/v1/claims")
@RequiredArgsConstructor
final class ClaimIntakeController {

    private final SubmitClaimUseCase submitClaimUseCase;
    private final ClaimWebMapper claimWebMapper;

    @PostMapping
    ResponseEntity<ClaimAcceptedResponse> submit(
        @RequestHeader("Idempotency-Key") @NotBlank String idempotencyKey,
        @RequestHeader(value = "X-Correlation-Id", required = false)
        UUID suppliedCorrelationId,
        @Valid @RequestBody SubmitClaimRequest request
    ) {
        UUID correlationId = suppliedCorrelationId == null
            ? UUID.randomUUID()
            : suppliedCorrelationId;

        SubmitClaimCommand command = claimWebMapper.toCommand(
            idempotencyKey,
            correlationId,
            request
        );
        ClaimAccepted result = submitClaimUseCase.submit(command);

        return ResponseEntity.accepted()
            .location(URI.create("/api/v1/claims/" + result.claimId()))
            .body(ClaimAcceptedResponse.from(result));
    }
}
```

15. Java implementation - transaction and outbox

The intake use case handles idempotency and creates the claim. The claim, idempotency record and outbox event are committed in the same local transaction.

SubmitClaimService.java

JAVA

```

@Service
@RequiredArgsConstructor
final class SubmitClaimService implements SubmitClaimUseCase {

    private final ClaimRepository claimRepository;
    private final IdempotencyRepository idempotencyRepository;
    private final OutboxRepository outboxRepository;
    private final RequestHasher requestHasher;

    @Transactional
    @Override
    public ClaimAccepted submit(SubmitClaimCommand command) {
        String requestHash = requestHasher.hash(command);

        return idempotencyRepository
            .find(command.channel(), command.idempotencyKey())
            .map(record -> reuseExistingResult(record, requestHash))
            .orElseGet(() -> createClaim(command, requestHash));
    }

    private ClaimAccepted createClaim(
        SubmitClaimCommand command,
        String requestHash
    ) {
        Claim claim = Claim.receive(command);
        claimRepository.save(claim);

        idempotencyRepository.save(
            IdempotencyRecord.forClaim(command, requestHash, claim)
        );
        outboxRepository.append(ClaimSubmitted.from(claim));

        return ClaimAccepted.from(claim);
    }

    private ClaimAccepted reuseExistingResult(
        IdempotencyRecord record,
        String requestHash
    ) {
        if (!record.requestHash().equals(requestHash)) {
            throw new IdempotencyConflictException(record.idempotencyKey());
        }
        return record.toClaimAccepted();
    }
}

```

Why this shape matters

- A repeated request with the same body returns the original result.
- The same key with a different body is rejected.
- The event cannot be lost between the database commit and message publication.
- The domain does not depend on AWS or Spring messaging classes.

15. Java implementation - domain and integrations

Claim.java

JAVA

```

public final class Claim {

    private ClaimStatus status;
    private String paymentReference;
    private Instant updatedAt;

    public void markPaymentCompleted(
        String receivedPaymentReference,
        Instant completedAt
    ) {
        if (status == ClaimStatus.PAID
            && receivedPaymentReference.equals(paymentReference)) {
            return; // The same callback is safe to process again.
        }

        if (status != ClaimStatus.PAYMENT_PENDING) {
            throw new
                InvalidClaimTransitionException(
                    status,
                    ClaimStatus.PAID
                );
        }

        status = ClaimStatus.PAID;
        paymentReference = receivedPaymentReference;
        updatedAt = completedAt;

        registerEvent(new ClaimPaid(
            id,
            receivedPaymentReference,
            completedAt
        ));
    }
}

```

PolicyManagerHttpAdapter.java

JAVA

```

@Component
@RequiredArgsConstructor
final class PolicyManagerHttpAdapter implements PolicyManagerPort {

    private final RestClient policyManagerClient;

    @Override
    public PolicyEligibilityResult checkEligibility(
        PolicyEligibilityCommand command
    ) {
        PolicyManagerResponse response = policyManagerClient.post()
            .uri("/api/v1/claim-eligibility-checks")
            .header("Idempotency-Key", command.idempotencyKey())
            .header("X-Correlation-Id", command.correlationId().toString())
            .body(PolicyManagerRequest.from(command))
            .retrieve()
            .body(PolicyManagerResponse.class);

        if (response == null) {
            throw new InvalidIntegrationResponseException(
                "Policy Manager returned an empty response"
            );
        }

        return response.toDomain();
    }
}

```

PaymentStatusEventHandler.java

JAVA

```

@Component
@RequiredArgsConstructor
final class PaymentStatusEventHandler {

    private final ClaimRepository claimRepository;
    private final ProcessedMessageRepository processedMessageRepository;

    @Transactional
    public void handle(PaymentStatusEvent event) {
        if (processedMessageRepository.exists("payment-status", event.eventId())) {
            return;
        }

        Claim claim = claimRepository.getRequired(event.claimId());
        claim.apply(event);

        claimRepository.save(claim);
        processedMessageRepository.markProcessed(
            "payment-status",
            event.eventId()
        );
    }
}

```

16. Dummy endpoints and local demonstration

The external stubs make the happy path and failure paths easy to demonstrate without access to real enterprise systems.

Dummy external endpoints

TEXT

```
# Client Registry
POST /api/v1/client-validations
200 OK
{"validationReference":"CV-10042","valid":true,"clientStatus":"ACTIVE","reasonCodes":[]}

# Policy Manager
POST /api/v1/claim-eligibility-checks
200 OK
{"validationReference":"PV-50042","eligible":true,"manualReviewRequired":true,"ruleVersion":"policy-rules-v1"}

# Payment System
POST /api/v1/payment-requests
202 Accepted
{"paymentReference":"PAY-852197","status":"ACCEPTED"}
```

Local demonstration

BASH

```
docker compose up -d postgres client-registry policy-manager payment-system
./gradlew clean test
./gradlew :applications:claims-service:bootRun

curl -X POST http://localhost:8080/api/v1/claims \
-H 'Content-Type: application/json' \
-H 'Idempotency-Key: demo-claim-001' \
--data @examples/claim-request.json
```

16.1 Failure modes supported by the stubs

SUCCESS, BUSINESS_REJECTION, TIMEOUT, TEMPORARY_FAILURE, DUPLICATE_CALLBACK, OUT_OF_ORDER_CALLBACK and AMBIGUOUS_PAYMENT_RESPONSE.

The local profile uses PostgreSQL and HTTP stubs. The AWS profile replaces the local workflow and messaging ports with Step Functions, EventBridge and SQS adapters.

17. Testing strategy

17.1 Test layers

Layer	What is tested	Tools
Domain unit tests	State transitions, priority, SLA rules, duplicate callbacks and business invariants	JUnit 5, AssertJ
Application tests	Use-case orchestration with mocked ports and controlled time	JUnit 5, Mockito
Persistence integration	Flyway migrations, constraints, outbox, optimistic locking and transaction behaviour	Testcontainers PostgreSQL
HTTP integration	Validation, security, error mapping and request/response contracts	Spring Boot Test, RestAssured
Consumer and provider contracts	Compatibility with Client Registry, Policy Manager and Payment System	OpenAPI checks or Pact
Architecture tests	Module rules and dependency direction	ArchUnit
End-to-end tests	Full process using external stubs	Docker Compose and Testcontainers

17.2 Minimum scenarios

- Valid straight-through claim.
- Invalid client.
- Inactive or ineligible policy.
- Manual approval.
- Manual rejection.
- Duplicate claim submission.
- Payment timeout and reconciliation.
- Duplicate payment callback.
- Out-of-order payment event.
- Downstream outage and recovery.
- Concurrent analyst and callback update.
- Critical claim under queue load.

Quality gate

A test is not useful just because it is green. The suite should assert business results, state transitions and side effects, not only HTTP status codes.

18. CI/CD and deployment

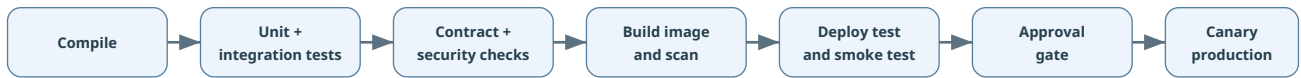


Figure 6. Proposed continuous-delivery flow.

18.1 Pipeline gates

- 1 Compile and run formatting checks.
- 2 Run unit, integration and architecture tests.
- 3 Validate OpenAPI and event contracts.
- 4 Run dependency, secret and static-code scans.
- 5 Build one immutable container image and scan it.
- 6 Deploy the same image to an isolated test environment.
- 7 Run migrations, contract checks and smoke tests.
- 8 Promote through an approval gate, then release with a canary.

18.2 Deployment controls

- Infrastructure is defined with AWS CDK or Terraform and reviewed like application code.
- Flyway migrations are backward compatible during rolling deployment.
- Feature flags separate workflow activation from code deployment.
- Rollback is automatic when health or business metrics move outside agreed limits.
- Production access is controlled, logged and time limited.
- Backups, restore tests and a documented recovery plan are part of the service.

18.3 Safe release measures

Infrastructure health is not enough. A release should also watch claim intake rate, workflow completion, payment failure rate, SLA warnings and queue age. A technically healthy service can still be wrong for the business.

19. Assumptions, questions and trade-offs

19.1 Assumptions

1. The Channel System can send HTTPS requests with an external reference and idempotency key.
2. The submitted JSON contains enough identifiers for client and policy checks.
3. Existing systems can expose APIs or can be wrapped by adapters.
4. The Payment System can support idempotency or a reliable lookup by a stable request key.
5. Some claims can use straight-through processing; others need an analyst.
6. The Claims Platform will become the only owner of claim workflow state.
7. Claim documents are outside the minimum build and would use encrypted object storage.
8. The exact SLA rules and retention periods will be confirmed by the business and compliance teams.

19.2 Questions I would confirm before build

- What are the SLA targets by claim type and priority?
- Which decisions can be automated?
- What evidence must be kept for a rejected claim?
- Can client and policy checks run in parallel?
- What is the Payment System's idempotency contract?
- How are reversals and payment corrections handled?
- Which roles can approve high-value claims?
- What are the data retention and residency rules?

19.3 Main trade-offs

Decision	Selected option	Trade-off
Application shape	Modular Spring Boot service	Lower delivery and operating cost now; extraction points remain.
Workflow	Step Functions orchestration	Clear long-running state; introduces AWS workflow dependency.
Database	Aurora PostgreSQL	Strong constraints and audit queries; more relational design work.
Messaging	EventBridge and SQS	Durable delivery and back-pressure; eventual consistency must be managed.
External validation	Bounded synchronous calls inside workflow steps	Simple request/response semantics; strict timeouts are required.
Payments	Idempotency plus reconciliation	Avoids an impossible cross-system transaction; accepts eventual completion.

20. Delivery plan, AI use and conclusion

20.1 Suggested implementation order

- 1 Agree the business states, reason codes, SLAs and ownership boundaries.
- 2 Build claim intake, validation, persistence, idempotency and status queries.
- 3 Add the outbox, workflow port and client/policy mock adapters.
- 4 Add analyst review and a full audit timeline.
- 5 Add payment idempotency, callback handling and reconciliation.
- 6 Add AWS infrastructure, priority queues, dashboards, alarms and runbooks.
- 7 Move from stubs to real adapters and release through shadow mode and a controlled claim group.

20.2 AI use

I used AI as a review and drafting assistant. It helped me challenge failure scenarios, compare architecture options and improve parts of the document. I reviewed the design, rewrote the final explanation in my own words, checked the code and kept responsibility for the technical decisions. I would be able to explain or change every part of the solution during an interview.

20.3 Conclusion

The proposed platform connects the existing systems without weakening their ownership. It accepts claims safely, coordinates client and policy checks, supports analyst decisions, creates payment requests only after approval and keeps one clear audit trail from submission to settlement.

The most important part of the design is not the number of AWS services. It is the way failures are handled: an accepted claim is durable, duplicate work is safe, uncertain payments are reconciled, urgent claims receive capacity, and operations can see what is happening.

Final design principle

A temporary downstream failure may delay a claim, but it must never lose an accepted claim, create a duplicate payment, or leave the business unable to explain what happened.

Completion check

Concise solution description	Included
Architecture and workflow diagrams	Included
Assumptions and trade-offs	Included
Integration and system-boundary recommendations	Included
End-to-end interaction	Included
Project template, sample Java code and dummy endpoints	Included
Reliability, security, SLA, testing and deployment	Included
AI use explanation	Included